

Lab 13 – Code Refactoring Using AI Suggestions

Task Description #1 (Refactoring – Removing Code Duplication)

- Task: Use AI to refactor a given Python script that contains multiple repeated code blocks.
- Instructions:
 - Prompt AI to identify duplicate logic and replace it with functions or classes.
 - Ensure the refactored code maintains the same output.
 - Add docstrings to all functions.
- Sample Legacy Code:

Legacy script with repeated logic

```
print("Area of Rectangle:", 5 * 10)
```

```
print("Perimeter of Rectangle:", 2 * (5 + 10))
```

```
print("Area of Rectangle:", 7 * 12)
```

```
print("Perimeter of Rectangle:", 2 * (7 + 12))
```

```
print("Area of Rectangle:", 10 * 15)
```

```
print("Perimeter of Rectangle:", 2 * (10 + 15))
```

- Expected Output:
 - Refactored code with a reusable function and no duplication.
 - Well documented code

Area of Rectangle :

```
week 13 > rectangle.py > ...
11 #write refactored code for the above code and identify the duplicate logic and correct it
12 def area_of_rectangle(length: float, breadth: float) -> float:
13     """
14     This function calculates the area of a rectangle given its length and breadth.
15     Parameters:
16     length (float): The length of the rectangle.
17     breadth (float): The breadth of the rectangle.
18     Returns:
19     float: The area of the rectangle.
20     """
21     return length * breadth
```

Perimeter of rectangle :

```
def perimeter_of_rectangle(length: float, breadth: float) -> float:
    """This function calculates the perimeter of a rectangle given its length and breadth.
    Parameters:
    length (float): The length of the rectangle.
    breadth (float): The breadth of the rectangle.
    Returns:
    float: The perimeter of the rectangle.
    """
    return 2 * (length + breadth)
```

Output :

```
31 print("Area of Rectangle:", area_of_rectangle(5, 10))
32 print("Perimeter of Rectangle:", perimeter_of_rectangle(5, 10))
33 print("Area of Rectangle:", area_of_rectangle(7, 12))
34 print("Perimeter of Rectangle:", perimeter_of_rectangle(7, 12))
35 print("Area of Rectangle:", area_of_rectangle(10, 15))
```

```
/AIAC/week 13/rectangle.py"
Area of Rectangle: 50
Perimeter of Rectangle: 30
Area of Rectangle: 84
Perimeter of Rectangle: 38
Area of Rectangle: 150
```

Task Description #2 (Refactoring – Extracting Reusable Functions)

- Task: Use AI to refactor a legacy script where multiple calculations are embedded directly inside the main code block.
- Instructions:
 - Identify repeated or related logic and extract it into reusable functions.
 - Ensure the refactored code is modular, easy to read, and documented with docstrings.
- Sample Legacy Code:

Legacy script with inline repeated logic

```
price = 250
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

```
price = 500
tax = price * 0.18
total = price + tax
print("Total Price:", total)
```

- Expected Output:
 - Code with a function `calculate_total(price)` that can be reused for multiple price inputs.
 - Well documented code

Refactored Code:

```

week 13 > total_price.py > ...
 9  print("Total Price:", total)
10  """
11  #write a refcatored code for the above code and use google style documentation for the explanation
12  def calculate_total(price):
13      """
14      This function calculates the total price of an item including tax.
15
16      Args:
17      |   price (float): The original price of the item.
18
19      Returns:
20      |   float: The total price of the item including tax.
21      """
22      tax = price * 0.18
23      total = price + tax
24      return total
25  print("Total Price:", calculate_total(250))
26  print("Total Price:", calculate_total(500))

```

Output:

```

al_price.py"
Total Price: 295.0
Total Price: 590.0
PS C:\Users\User\OneDrive\Desktop\AIAC>

```

Task Description #3: Refactoring Using Classes and Methods (Eliminating Redundant Conditional Logic)

Refactor a Python script that contains repeated ~~if-elif-else~~ grading logic by implementing a structured, object-oriented solution using a class and a method.

Problem Statement

The given script contains duplicated conditional statements used to assign grades based on student marks. This redundancy violates clean code principles and reduces maintainability.

You are required to refactor the script using a class-based design to improve modularity, reusability, and readability while preserving the original grading logic.

Mandatory Implementation Requirements

1. Class Name: GradeCalculator
2. Method Name: calculate_grade(self, marks)
3. The method must:
 - o Accept marks as a parameter.
 - o Return the corresponding grade as a string.
 - o The grading logic must strictly follow the conditions below:
 - $\text{Marks} \geq 90 \text{ and } \leq 100 \rightarrow \text{"Grade A"}$
 - $\text{Marks} \geq 80 \rightarrow \text{"Grade B"}$
 - $\text{Marks} \geq 70 \rightarrow \text{"Grade C"}$
 - $\text{Marks} \geq 40 \rightarrow \text{"Grade D"}$
 - $\text{Marks} \geq 0 \rightarrow \text{"Fail"}$

Note: Assume marks are within the valid range of 0 to 100.

4. Include proper docstrings for:
 - o The class
 - o The method (with parameter and return descriptions)
5. The method must be reusable and called multiple times without rewriting conditional logic.

• Given code:

```
marks = 85
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
```

```

else:
    print("Grade C")
marks = 72
if marks >= 90:
    print("Grade A")
elif marks >= 75:
    print("Grade B")
else:
    print("Grade C")

```

Expected Output:

- Define a class named GradeCalculator.
- Implement a method calculate_grade(self, marks) inside the class.

Refactored Code:

```

week 13 > grade.py > GradeCalculator > calculate_grade
30 class GradeCalculator:
31     """
32     A class to calculate grades based on marks.
33     """
34     def calculate_grade(self, marks):
35         """
36         Calculates the grade based on the given marks.
37
38         Args:
39         |     marks (int): The marks obtained by a student.
40
41         Returns:
42         |     str: The grade corresponding to the marks.
43         """
44         if marks >= 90 and marks <= 100:
45             return "Grade A"
46         elif marks >= 80:
47             return "Grade B"
48         elif marks >= 70:
49             return "Grade C"
50         elif marks >= 40:
51             return "Grade D"
52         else:
53             return "Fail"

```

Output:

```
# Example usage
grade_calculator = GradeCalculator()
print(grade_calculator.calculate_grade(85)) # Output: Grade B
print(grade_calculator.calculate_grade(72)) # Output: Grade C
print(grade_calculator.calculate_grade(95)) # Output: Grade A
print(grade_calculator.calculate_grade(65)) # Output: Grade D
print(grade_calculator.calculate_grade(30)) # Output: Fail
```

```
de.py"
Grade B
Grade C
Grade A
Grade D
Fail
```

Task Description #4 (Refactoring – Converting Procedural Code to Functions)

- **Task:** Use AI to refactor procedural input–processing logic into functions.

Instructions:

- o Identify input, processing, and output sections.
- o Convert each into a separate function.
- o Improve code readability without changing behavior.

- Sample Legacy Code:

```
num = int(input("Enter number: "))
square = num * num
print("Square:", square)
```

- Expected Output:

- o Modular code using functions like `get_input()`, `calculate_square()`, and `display_result()`.

`get_input()` :

```

week 13 > square.py > get_input
6 # write a refactor and Modular code using functions like get_input(), calculate_s
7 def get_input():|
8     """
9     This function prompts the user to enter a number and returns it as an integer.
10
11     Returns:
12     |   int: The number entered by the user.
13     """
14     return int(input("Enter number: "))

```

calculate_square() :

```

week 13 > square.py > calculate_square
14     return int(input("Enter number: "))
15 def calculate_square(num):
16     """This function calculates the square of a given number.
17
18     Args:
19     |   num (int): The number to be squared.
20
21     Returns:
22     |   int: The square of the given number.
23     """
24     return num * num

```

Display_result :

```

week 13 > square.py > calculate_square
25 def display_result(square):
26     """This function displays the calculated square.
27
28     Args:
29     |   square (int): The calculated square.
30     """
31     print("Square:", square)

```


Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

Focus Areas:

- o Object-Oriented principles

- o Encapsulation

Legacy Code:

```
salary = 50000
```

```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes.

```
week 13 > salary.py > EmployeeSalaryCalculator > calculate_net_salary

9  class EmployeeSalaryCalculator:
10     """A class to calculate the net salary of an employee after tax deductions."""
11
12     def __init__(self, salary):
13         """
14         Initializes the EmployeeSalaryCalculator with the given salary.
15
16         Args:
17             salary (float): The gross salary of the employee.
18         """
19         self.salary = salary
20
21     def calculate_tax(self):
22         """
23         Calculates the tax based on a fixed tax rate.
24
25         Returns:
26             float: The amount of tax to be deducted.
27         """
28         tax_rate = 0.2
29         return self.salary * tax_rate
30
31     def calculate_net_salary(self):
32         """
33         Calculates the net salary after deducting tax.
34
35         Returns:
36             float: The net salary of the employee.
37         """
```

```

38         tax = self.calculate_tax()
39         net_salary = self.salary - tax
40         return net_salary
41     # Example usage
42     if __name__ == "__main__":
43         salary = 50000
44         calculator = EmployeeSalaryCalculator(salary)
45         net_salary = calculator.calculate_net_salary()
46         print(net_salary)

```

Output:

```

PS C:\Users\User\OneDrive\Desktop\AIAC>
ary.py"
40000.0

```

Task 6 (Optimizing Search Logic)

- **Task:** Refactor inefficient linear searches using appropriate data structures.

- Focus Areas:

- o Time complexity

- o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]
```

```
name = input("Enter username: ")
```

```
found = False
```

```
for u in users:
```

```
    if u == name:
```

```
        found = True
```

```
print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

- o Use of sets or dictionaries with complexity justification

```

week 13 > username.py > UserAccessControl
10 # Refactored Code:Expected Outcome: Use of sets or dictionaries with complexity justification
11 class UserAccessControl:
12     """A class to manage user access control using a set for efficient lookups."""
13
14     def __init__(self, users):
15         """
16         Initializes the UserAccessControl with a list of users.
17
18         Args:
19             users (list): A list of usernames that have access.
20         """
21         self.users = set(users) # Using a set for O(1) average time complexity for lookups
22
23     def check_access(self, username):
24         """
25         Checks if the given username has access.
26
27         Args:
28             username (str): The username to check for access.
29         Returns:
30             bool: True if access is granted, False otherwise.
31         """
32         return username in self.users # O(1) average time complexity for lookups
33
34 # Example usage
35 if __name__ == "__main__":
36     users = ["admin", "guest", "editor", "viewer"]
37     access_control = UserAccessControl(users)
38     name = input("Enter username: ")
39     if access_control.check_access(name):
40         print("Access Granted")
41     else:
42         print("Access Denied")

```

Output:

```

PS C:\Users\User\OneDrive\Desktop\AIAC>
rname.py"
Enter username: admin
Access Granted
PS C:\Users\User\OneDrive\Desktop\AIAC>
rname.py"
Enter username: srijith
Access Denied

```

Task 7 – Refactoring the Library Management System

Problem Statement

You are provided with a poorly structured Library Management script that:

- Contains repeated conditional logic
- Does not use reusable functions
- Lacks documentation
- Uses print-based procedural execution
- Does not follow modular programming principles

Your task is to refactor the code into a proper format

1. Create a module `library.py` with functions:
 - `add_book(title, author, isbn)`
 - `remove_book(isbn)`
 - `search_book(isbn)`
2. Insert triple quotes under each function and let Copilot complete the docstrings.
3. Generate documentation in the terminal.
4. Export the documentation in HTML format.
5. Open the file in a browser.

Given Code

```
# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}
# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")
```

```

# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"

if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")

# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")

# Searching again
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

```

week 13 > library.py > ...

```
43
44 class Library:
45     """A class to manage a library system with functionalities to add, search, and remove books."""
46
47     def __init__(self):
48         """Initializes the Library with an empty database."""
49         self.library_db = {}
50
51     def add_book(self, title, author, isbn):
52         """
53         Adds a book to the library database.
54
55         Args:
56             title (str): The title of the book.
57             author (str): The author of the book.
58             isbn (str): The ISBN number of the book.
59
60         Returns:
61             str: A message indicating whether the book was added or already exists.
62         """
63         if isbn not in self.library_db:
64             self.library_db[isbn] = {"title": title, "author": author}
65             return "Book added successfully."
66         else:
67             return "Book already exists."
```

week 13 > library.py > ...

```
44 class Library:
68
69     def search_book(self, isbn):
70         """
71         Searches for a book in the library database by its ISBN.
72
73         Args:
74             isbn (str): The ISBN number of the book to search for.
75
76         Returns:
77             str: A message indicating whether the book was found or not.
78         """
79         if isbn in self.library_db:
80             return f"Book Found: {self.library_db[isbn]}"
81         else:
82             return "Book not found."
```

```

week 13 > library.py > ...
44 class Library:
84     def remove_book(self, isbn):
85         """
86         Removes a book from the library database by its ISBN.
87
88         Args:
89             isbn (str): The ISBN number of the book to remove.
90         Returns:
91             str: A message indicating whether the book was removed or not found.
92         """
93         if isbn in self.library_db:
94             del self.library_db[isbn]
95             return "Book removed successfully."
96         else:
97             return "Book not found."
98
99 # Example usage
100 if __name__ == "__main__":
101     library = Library()
102     print(library.add_book("Python Basics", "John Doe", "101"))
103     print(library.add_book("AI Fundamentals", "Jane Smith", "102"))
104     print(library.search_book("101"))
105     print(library.remove_book("101"))
106     print(library.search_book("101"))

```

Output:

```

library.py
Book added successfully.
Book added successfully.
Book Found: {'title': 'Python Basics', 'author': 'John Doe'}
Book removed successfully.
library.py
PS C:\Users\User\OneDrive\Desktop\AIAC> cd "week 13"
PS C:\Users\User\OneDrive\Desktop\AIAC\week 13> python -m pydoc library
Help on module library:

NAME
    library

DESCRIPTION
    Given Code
    # Library Management System (Unstructured Version)
    # This code needs refactoring into a proper module with documentation.
    library_db = {}
    # Adding first book
    title = "Python Basics"
    author = "John Doe"
    isbn = "101"
    if isbn not in library_db:
        library_db[isbn] = {"title": title, "author": author}

```

```

BOOK NOT FOUND.
PS C:\Users\User\OneDrive\Desktop\AIAC> cd "week 13"
PS C:\Users\User\OneDrive\Desktop\AIAC\week 13> python -m pydoc -w library
wrote library.html
PS C:\Users\User\OneDrive\Desktop\AIAC\week 13> python -m pydoc -p 1234
Server ready at http://localhost:1234/
Server commands: [b]rowser, [q]uit
server> b

```

library

Given Code

```

# Library Management System (Unstructured Version)
# This code needs refactoring into a proper module with documentation.
library_db = {}
# Adding first book
title = "Python Basics"
author = "John Doe"
isbn = "101"
if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")
# Adding second book (duplicate logic)
title = "AI Fundamentals"
author = "Jane Smith"
isbn = "102"
if isbn not in library_db:
    library_db[isbn] = {"title": title, "author": author}
    print("Book added successfully.")
else:
    print("Book already exists.")
# Searching book (repeated logic structure)
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")
# Removing book (again repeated pattern)
isbn = "101"
if isbn in library_db:
    del library_db[isbn]
    print("Book removed successfully.")
else:
    print("Book not found.")
# Searching again
isbn = "101"
if isbn in library_db:
    print("Book Found:", library_db[isbn])
else:
    print("Book not found.")

```


Task 8– Fibonacci Generator

Write a program to generate Fibonacci series up to n.

The initial code has:

- Global variables.
- Inefficient loop.
- No functions or modularity.

Task for Students:

- Refactor into a clean reusable function (`generate_fibonacci`).
- Add docstrings and test cases.
- Compare AI-refactored vs original.

Bad Code Version:

```
# fibonacci bad version
n=int(input("Enter limit: "))
a=0
b=1
print(a)
print(b)
for i in range(2,n):
    c=a+b
    print(c)
    a=b
    b=c
```

#Refactored Code :

```
week 13 > 📁 fibonacci.py > ...
15  #write refactored code for above fibonacci code with proper documentation and
16  def fibonacci(n):
17      """
18      Generate Fibonacci sequence up to the nth term.
19
20      Args:
21      |   n (int): The number of terms in the Fibonacci sequence to generate.
22
23      Returns:
24      |   list: A list containing the Fibonacci sequence up to the nth term.
25      """
```

```

26     if n <= 0:
27         return []
28     elif n == 1:
29         return [0]
30     elif n == 2:
31         return [0, 1]
32
33     sequence = [0, 1]
34     for i in range(2, n):
35         next_term = sequence[i - 1] + sequence[i - 2]
36         sequence.append(next_term)
37
38     return sequence
39 # Test cases
40 if __name__ == "__main__":
41     print(fibonacci(10)) # Expected output: [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
42     print(fibonacci(0)) # Expected output: []
43     print(fibonacci(1)) # Expected output: [0]
44     print(fibonacci(2)) # Expected output: [0, 1]
45     print(fibonacci(5)) # Expected output: [0, 1, 1, 2, 3]
46     print(fibonacci(-1)) # Expected output: []

```

Output :

```

onacci.py"
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
[]
[0]
[0, 1]
[0, 1, 1, 2, 3]
[]
PS C:\Users\User\OneDrive\Desktop\AIAC>

```

Task 9 – Twin Primes Checker

Twin primes are pairs of primes that differ by 2 (e.g., 11 and 13, 17 and 19).

The initial code has:

- Inefficient prime checking.
- No functions.
- Hardcoded inputs.

Task for Students:

- Refactor into `is_prime(n)` and `is_twin_prime(p1, p2)`.
- Add docstrings and optimize.
- Generate a list of twin primes in a given range using AI.

Bad Code Version:

```
# twin primes bad version
a=11
b=13
fa=0
for i in range(2,a):
    if a%i==0:
        fa=1
fb=0
for i in range(2,b):
    if b%i==0:
        fb=1
if fa==0 and fb==0 and abs(a-b)==2:
    print("Twin Primes")
else:
    print("Not Twin Primes")
```

week 13 > prime.py > ...

```
18 """
19 # Refactored Code with Proper Documentation and Modular Structure
20 def is_prime(num):
21     """
22     Check if a number is prime.
23
24     Args:
25         num (int): The number to check for primality.
26     Returns:
27         bool: True if the number is prime, False otherwise.
28     """
29     if num <= 1:
30         return False
31     for i in range(2, int(num**0.5) + 1):
32         if num % i == 0:
33             return False
34     return True
```

```

35 def are_twin_primes(a, b):
36     """Check if two numbers are twin primes.
37     Args:
38         a (int): The first number.
39         b (int): The second number.
40     Returns:
41         bool: True if the numbers are twin primes, False otherwise.
42     """
43     return is_prime(a) and is_prime(b) and abs(a - b) == 2
44 # Test cases
45 if __name__ == "__main__":
46     print(are_twin_primes(11, 13)) # Expected output: True
47     print(are_twin_primes(17, 19)) # Expected output: True
48     print(are_twin_primes(4, 6))   # Expected output: False
49     print(are_twin_primes(29, 31)) # Expected output: True
50     print(are_twin_primes(10, 12)) # Expected output: False

```

#Output :

```

● PS C:\Users\User\OneDrive\Desktop\AIAC> &
me.py"
True
True
False
True
False
○ PS C:\Users\User\OneDrive\Desktop\AIAC>

```

Task 10 – Refactoring the Chinese Zodiac Program

Objective

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

The current program reads a year from the user and prints the corresponding Chinese Zodiac sign. However, the implementation contains repetitive conditional logic, lacks modular design, and does not follow clean coding principles.

Your task is to refactor the code to improve readability, maintainability, and structure.

Chinese Zodiac Cycle (Repeats Every 12 Years)

1. Rat
2. Ox
3. Tiger
4. Rabbit
5. Dragon

6. Snake
7. Horse
8. Goat (Sheep)
9. Monkey
10. Rooster
11. Dog
12. Pig

Chinese Zodiac Program (Unstructured Version)

This code needs refactoring.

```
year = int(input("Enter a year: "))
if year % 12 == 0:
    print("Monkey")
elif year % 12 == 1:
    print("Rooster")
elif year % 12 == 2:
    print("Dog")
elif year % 12 == 3:
    print("Pig")
elif year % 12 == 4:
    print("Rat")
elif year % 12 == 5:
    print("Ox")
elif year % 12 == 6:
    print("Tiger")
elif year % 12 == 7:
    print("Rabbit")
elif year % 12 == 8:
    print("Dragon")
elif year % 12 == 9:
    print("Snake")
elif year % 12 == 10:
    print("Horse")
elif year % 12 == 11:
    print("Goat")
```

You must:

1. Create a reusable function: `get_zodiac(year)`
2. Replace the if-elif chain with a cleaner structure (e.g., list or dictionary).

- J / -

```

week 13 > 🐘 chinese.py > ...
38 # Refactored Code with Proper Documentation and Modular Structure
39 def get_zodiac(year):
40     """
41     Get the Chinese Zodiac sign for a given year.
42
43     Args:
44         year (int): The year for which to determine the Chinese Zodiac sign.
45     Returns:
46         str: The Chinese Zodiac sign corresponding to the given year.
47     """
48     zodiac_signs = [
49         "Monkey", "Rooster", "Dog", "Pig", "Rat", "Ox",
50         "Tiger", "Rabbit", "Dragon", "Snake", "Horse", "Goat"
51     ]
52     return zodiac_signs[year % 12]
53 if __name__ == "__main__":
54     try:
55         year_input = int(input("Enter a year: "))
56         zodiac_sign = get_zodiac(year_input)
57         print(f"The Chinese Zodiac sign for the year {year_input} is: {zodiac_sign}")
58     except ValueError:
59         print("Invalid input. Please enter a valid year.")

```

```
PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/Users/User/AppData/Local/Programs/Python/Python39-32/python.exe nese.py
Enter a year: 2012
The Chinese Zodiac sign for the year 2012 is: Dragon
```

Refactor the given poorly structured Python script into a clean, modular, and reusable implementation.

For example:

- $18 \rightarrow 1 + 8 = 9 \rightarrow 18 \div 9 = 2$  (Harshad Number)
- $19 \rightarrow 1 + 9 = 10 \rightarrow 19 \div 10 \neq \text{integer}$  (Not Harshad)

Problem Statement

The current implementation:

- Mixes logic and input handling
- Uses redundant variables
- Does not use reusable functions properly
- Returns print statements instead of boolean values
- Lacks documentation

You must refactor the code to follow clean coding principles.

Harshad Number Checker (Unstructured Version)

```
num = int(input("Enter a number: "))
```

```
temp = num
```

```
sum_digits = 0
```

```
digit = temp % 10
```

```
sum_digits = sum_digits + digit
```

```
temp = temp // 10
```

```
if sum_digits != 0:
```

```
    if num % sum_digits == 0:
```

```
        print("True")
```

```
    else:
```

```
        print("False")
```

```
else:
```

```
    print("False")
```

You must:

1. Create a reusable function: is_harshad(number)
2. The function must:
 - Accept an integer parameter.
 - Return True if the number is divisible by the sum of its digits.
 - Return False otherwise.
3. Separate user input from core logic.
4. Add proper docstrings.
5. Improve readability and maintainability.
6. Ensure the program handles edge cases (e.g., 0, negative numbers).

#Refactored Code:

```

week 13 > harshad.py > ...
19 def is_harshad(num):
20     """
21     Check if a number is a Harshad number.
22
23     A Harshad number (or Niven number) is an integer that is divisible by the sum of its digits.
24
25     Args:
26     |   num (int): The number to check.
27     Returns:
28     |   bool: True if the number is a Harshad number, False otherwise.
29     """
30     if num <= 0:
31         return False
32
33     sum_digits = sum(int(digit) for digit in str(num))
34
35     if sum_digits == 0:
36         return False
37
38     return num % sum_digits == 0

```

Output:

```

39 # Test cases
40 if __name__ == "__main__":
41     test_numbers = [18, 19, 21, 0, -5]
42     for number in test_numbers:
43         result = is_harshad(number)
44         print(f"{number} is a Harshad number: {result}")
45

```

```

• PS C:\Users\User\OneDrive\Desktop\AIAC> & shad.py
18 is a Harshad number: True
19 is a Harshad number: False
21 is a Harshad number: True
0 is a Harshad number: False
-5 is a Harshad number: False
• PS C:\Users\User\OneDrive\Desktop\AIAC>

```

Task 12 – Refactoring the Factorial Trailing Zeros Program

Refactor the given poorly structured Python script into a clean, modular, and efficient implementation.

The program calculates the number of trailing zeros in $n!$ (factorial of n).

Problem Statement

The current implementation:

- Calculates the full factorial (inefficient for large n)
- Mixes input handling with business logic
- Uses print statements instead of return values
- Lacks modular structure and documentation

You must refactor the code to improve efficiency, readability, and maintainability.

Factorial Trailing Zeros (Unstructured Version)

```
n = int(input("Enter a number: "))
```

```
fact = 1
```

```
i = 1
```

```
while i <= n:
```

```
    fact = fact * i
```

```
    i = i + 1
```

```
count = 0
```

```
while fact % 10 == 0:
```

```
    count = count + 1
```

```
    fact = fact // 10
```

```
print("Trailing zeros:", count)
```

You must:

1. Create a reusable function: `count_trailing_zeros(n)`
2. The function must:
 - o Accept a non-negative integer `n`.
 - o Return the number of trailing zeros in `n!`.
3. Do NOT compute the full factorial.
4. Use an optimized mathematical approach (count multiples of 5).
5. Add proper docstrings.
6. Separate user interaction from core logic.
7. Handle edge cases (e.g., negative numbers, zero).

week 13 >  trailing.py > ...

```
14
15 # Refactored Code with Proper Documentation and Modular Structure
16 def count_trailing_zeros(n):
17     """
18     Count the number of trailing zeros in the factorial of a given number.
19
20     Args:
21         n (int): The number for which to calculate the factorial and count trailing zeros.
22     Returns:
23         int: The number of trailing zeros in n!.
24     """
25     if n < 0:
26         raise ValueError("Input must be a non-negative integer.")
27
28     count = 0
29     power_of_5 = 5
30     while n >= power_of_5:
31         count += n // power_of_5
32         power_of_5 *= 5
33
34     return count
```

#Output :

```

35 # Test cases
36 if __name__ == "__main__":
37     test_numbers = [0, 5, 10, 25, 100]
38     for number in test_numbers:
39         zeros = count_trailing_zeros(number)
40         print(f"The number of trailing zeros in {number}! is: {zeros}")

```

```

• PS C:\Users\User\OneDrive\Desktop\AIAC> & C:/U
illing.py"
The number of trailing zeros in 0! is: 0
The number of trailing zeros in 5! is: 1
The number of trailing zeros in 10! is: 2
The number of trailing zeros in 25! is: 6
The number of trailing zeros in 100! is: 24

```

Test Cases Design

Task 13 (Collatz Sequence Generator – Test Case Design)

- Function: Generate Collatz sequence until reaching 1.
- Test Cases to Design:
 - Normal: $6 \rightarrow [6, 3, 10, 5, 16, 8, 4, 2, 1]$
 - Edge: $1 \rightarrow [1]$
 - Negative: -5
 - Large: 27 (well-known long sequence)
 - Requirement: Validate correctness with pytest.

Explanation:

We need to write a function that:

- Takes an integer n as input.
- Generates the Collatz sequence (also called the $3n+1$ sequence).
- The rules are:
 - If n is even $\rightarrow \text{next} = n / 2$.
 - If n is odd $\rightarrow \text{next} = 3n + 1$.
- Repeat until we reach 1.
- Return the full sequence as a list.

Example

Input: 6

Steps:

- 6 (even $\rightarrow 6/2 = 3$)
- 3 (odd $\rightarrow 3*3+1 = 10$)
- 10 (even $\rightarrow 10/2 = 5$)
- 5 (odd $\rightarrow 3*5+1 = 16$)
- 16 (even $\rightarrow 16/2 = 8$)
- 8 (even $\rightarrow 8/2 = 4$)
- 4 (even $\rightarrow 4/2 = 2$)
- 2 (even $\rightarrow 2/2 = 1$)

Output:

[6, 3, 10, 5, 16, 8, 4, 2, 1]

week 13 > Collatz.py > ...

```
1  #write a python program Collatz Sequence
2  def collatz_sequence(n):
3      """
4      Generate the Collatz sequence for a given number n.
5
6      The Collatz sequence is defined as:
7      - If n is even, divide it by 2.
8      - If n is odd, multiply by 3 and add 1.
9      - Continue until reaching 1.
10
11     Args:
12     |     n (int): The starting number for the Collatz sequence.
13
14     Returns:
15     |     list: The Collatz sequence as a list of integers.
16     """
17     if n <= 0:
18         raise ValueError("Input must be a positive integer.")
19
20     sequence = [n]
21     while n != 1:
22         if n % 2 == 0:
23             n = n // 2
24         else:
25             n = 3 * n + 1
26         sequence.append(n)
27
28     return sequence
```

#Output :

```
29
30 # Example usage
31 v if __name__ == "__main__":
32 v     try:
33         number = int(input("Enter a positive integer: "))
34         result = collatz_sequence(number)
35         print(f"Collatz sequence for {number}: {result}")
36 v     except ValueError as e:
37         print(f"Error: {e}")
```

latz.py"

Enter a positive integer: 6

Collatz sequence for 6: [6, 3, 10, 5, 16, 8, 4, 2, 1]

PS C:\Users\User\OneDrive\Desktop\AIAC> █

Task 14 (Lucas Number Sequence – Test Case Design)

- Function: Generate Lucas sequence up to n terms.

(Starts with 2,1, then $F_n = F_{n-1} + F_{n-2}$)

- Test Cases to Design:
 - Normal: $5 \rightarrow [2, 1, 3, 4, 7]$
 - Edge: $1 \rightarrow [2]$
 - Negative: $-5 \rightarrow \text{Error}$
 - Large: 10 (last element = 76).
- Requirement: Validate correctness with pytest.

```
week 13 > Lucas.py > ...
1  #write a python program for Lucas Sequence use pytest
2  def lucas(n):
3      """
4      Generate the Lucas sequence up to the nth term.
5
6      Args:
7      |   n (int): The number of terms in the Lucas sequence to generate.
8
9      Returns:
10     |   list: A list containing the Lucas sequence up to the nth term.
11     """
12     if n <= 0:
13         return []
14     elif n == 1:
15         return [2]
16     elif n == 2:
17         return [2, 1]
18
19     sequence = [2, 1]
20     for i in range(2, n):
21         next_term = sequence[i - 1] + sequence[i - 2]
22         sequence.append(next_term)
23
24     return sequence
25
26 # Test cases using pytest
27 def test_lucas():
28     assert lucas(0) == []
29     assert lucas(1) == [2]
30     assert lucas(2) == [2, 1]
31     assert lucas(5) == [2, 1, 3, 4, 7]
32     assert lucas(10) == [2, 1, 3, 4, 7, 11, 18, 29, 47, 76]
```

#Output :

```

PS C:\Users\User\OneDrive\Desktop\AIAC> cd "week 13"
PS C:\Users\User\OneDrive\Desktop\AIAC\week 13> python -m pytest Lucas.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\User\OneDrive\Desktop\AIAC\week 13
collected 1 item

Lucas.py .

```

Task 15 (Vowel & Consonant Counter – Test Case Design)

- Function: Count vowels and consonants in string.
- Test Cases to Design:
 - Normal: "hello" → (2,3)
 - Edge: "" → (0,0)
 - Only vowels: "aeiou" → (5,0)

Large: Long text

- Requirement: Validate correctness with pytest.

```

week 13 > vowel.py > test_count_vowels_and_consonants
1  #write a python program to Vowel & Consonant Counter and write test cases using pytest
2  def count_vowels_and_consonants(text):
3      """
4      Count the number of vowels and consonants in a given text.
5
6      Args:
7      |   text (str): The input string to analyze.
8      Returns:
9      |   tuple: A tuple containing the count of vowels and consonants (vowels_count, consonants_count).
10     """
11     vowels = "aeiouAEIOU"
12     vowels_count = 0
13     consonants_count = 0
14
15     for char in text:
16         if char.isalpha(): # Check if the character is an alphabet
17             if char in vowels:
18                 vowels_count += 1
19             else:
20                 consonants_count += 1
21
22     return vowels_count, consonants_count

```

#Output :

```

PS C:\Users\User\OneDrive\Desktop\AIAC\week 13> & C:/Users/User/AppData/Local/Python/python
k 13/vowel.py"
PS C:\Users\User\OneDrive\Desktop\AIAC\week 13> python -m pytest vowel.py
===== test session starts =====
platform win32 -- Python 3.14.2, pytest-9.0.2, pluggy-1.6.0
rootdir: C:\Users\User\OneDrive\Desktop\AIAC\week 13
collected 1 item

vowel.py .

===== 1 passed in 0.12s =====

```